

PRÁCTICA 5 – ANEXO

IMPLEMENTACIÓN DE FUNCIONES EN MATLAB PARA LA APLICACIÓN DE TÉCNICAS DE CLASIFICACIÓN

Ejercicio:

Aplicando las funciones descritas en este documento, diseña clasificadores Naive-Bayes Gaussiano, LDA, QDA y KNN en el conjunto de entrenamiento y calcula su tasa de acierto en el conjunto de prueba en los conjuntos de datos facilitados.

Observaciones:

- En los casos LDA y QDA, se debe representar en el espacio de características, junto con los datos de entrenamiento, la frontera de separación de los clasificadores.
- En el caso del KNN, se debe trabajar con el valor de k óptimo, calculado mediante un proceso *hold-out* con división 70/30 (ver ayuda de programación en observación del ejercicio 3.b, del boletín de ejercicios).

Ayuda de programación: división de un conjunto de entrenamiento XTrain-YTrain en dos subconjuntos: entrenamiento reducido (70% de los datos seleccionados de forma aleatoria), XTrain2-YTrain2, y validación (30% de los datos restantes):

```
numDatos = size(XTrain,1);
porcentajeTrain2 = 0.7;
numDatosTrain2 = round(porcentajeTrain2*numDatos);
numerosMuestrasTrain2 = randsample(numDatos,numDatosTrain2);
numerosMuestrasVal = find(not(ismember(1:numDatos,numerosMuestrasTrain2)));

% Conjunto de Train2
XTrain2 = XTrain(numerosMuestrasTrain2,:);
YTrain2 = YTrain(numerosMuestrasTrain2);
% Conjunto de Val
XVal = XTrain(numerosMuestrasVal,:);
YVal = YTrain(numerosMuestrasVal);
```

TÉCNICAS DE CLASIFICACIÓN: FUNCIONES A IMPLEMENTAR

CLASIFICADOR NAIVE-BAYES GAUSSIANO

➤ **Funciones de ajuste (cálculo de información necesaria para la aplicación del clasificador) y aplicación del clasificador (clasificación multiclase y variables de entrada X p-dimensional):**

- **Función** `funcion_ajusta_NBG`:

Dado un conjunto de datos X-Y, la función calcula los valores medio y desviaciones típica de cada variable de entrada en cada clase, así como las probabilidades a priori para cada una de las clases del problema (dadas por los valores distintos de Y).

```
[medias, desviaciones, probabilidadPriori] = ...  
funcion_ajusta_NBG (X,Y);
```

- `medias`: almacena los valores medios de cada variable de entrada de cada clase (tantas filas como variables de entrada haya, tantas columnas como clases).
- `desviaciones`: almacena las desviaciones típicas de cada variable de entrada de cada clase (tantas filas como variables de entrada haya, tantas columnas como clases).
- `probabilidadPriori`: vector fila que almacena en cada columna la probabilidad a priori de cada clase.

- **Función** `funcion_aplica_NBG`:

Esta función utiliza la información generada por la función anterior para predecir la clase del conjunto de instancias dadas por X, Y_NBG. Los valores de codificación que utiliza para asignar las clases son los que se especifican en la variable de entrada `valoresClases`. Además, la función devuelve el resultado de la evaluación de las funciones de decisión del clasificador para cada instancia de X que motiva la predicción de la clase.

```
[Y_NBG, d] = funcion_aplica_NBG (...  
X, medias, desviaciones, probabilidadPriori, valoresClases);
```

CLASIFICADOR QDA:

➤ **Funciones de ajuste (cálculo de información necesaria para la aplicación del clasificador) y aplicación del clasificador (clasificación multiclase y variables de entrada X p-dimensional):**

- **Función** `funcion_ajusta_QDA`:

Dado un conjunto de datos X-Y, la función calcula los vectores de medias, matrices de covarianzas y probabilidades a priori para cada una de las clases del problema (dadas por los valores distintos de Y).

```
[vectorMedias, matricesCovarianzas, probabilidadPriori] = ...  
    funcion_ajusta_QDA (X,Y);
```

- `vectorMedias`: almacena los vectores de medias de cada clase (tantas filas como variables de entrada haya, tantas columnas como clases).
- `matricesCovarianzas`: almacena las matrices covarianza de cada clase. La tercera dimensión se utiliza para almacenar la matriz de covarianza de la clase en cuestión.
- `probabilidadPriori`: vector fila que almacena en cada columna la probabilidad a priori de cada clase.

- **Función** `funcion_aplica_QDA`:

Esta función utiliza la información generada por la función anterior para predecir la clase del conjunto de instancias dadas por X, Y_QDA. Los valores de codificación que utiliza para asignar las clases son los que se especifican en la variable de entrada `valoresClases`. Además, la función devuelve el resultado de la evaluación de las funciones de decisión del clasificador QDA para cada instancia de X que motiva la predicción de la clase.

```
[Y_QDA,d] = funcion_aplica_QDA (...  
X, vectorMedias, matricesCovarianzas, probabilidadPriori, valoresClases);
```

➤ **Función de diseño (cálculo de funciones de decisión y discriminante en lenguaje simbólico) para el caso específico de clasificación binaria y espacios de características de dos o tres dimensiones:**

- **Función** `funcion_calcula_d1_d2_d12_QDA_2Clases_2_3_Dim`:

Esta función es específica para conjuntos de datos X-Y compuestos por 2 o 3 variables de entrada y 2 clases. La función calcula las funciones de decisión `d1`, `d2`, y la función discriminante, `d12`, para una instancia X definida de forma simbólica. De esta forma, es posible representar la frontera de separación del clasificador $d12 = 0$, por medio de la función de Matlab `fimplicit` o `fimplicit3`, según se trabaje en dos o tres dimensiones, respectivamente.

```
[d1, d2, d12] = funcion_calcula_d1_d2_d12_QDA_2Clases_2_3_Dim(X, Y)
```

CLASIFICADOR LDA:

- **Funciones de ajuste (cálculo de información necesaria para la aplicación del clasificador) y aplicación general del clasificador (clasificación multiclase y variables de entrada X p-dimensional):**

- **Funciones** `funcion_ajusta_LDA` y `funcion_aplica_LDA`:

Funciones equivalentes a las dos anteriores correspondientes al clasificador QDA pero, en este caso, relativas al clasificador LDA.

```
[vectorMedias, matrizCovarianza, probabilidadPriori] = ...  
    funcion_ajusta_LDA (X,Y);  
  
[Y_LDA,d] = funcion_aplica_LDA (...  
X, vectorMedias,matrizCovarianza, probabilidadPriori, valoresClases);
```

- **Funciones de diseño (cálculo de funciones de decisión y discriminante en lenguaje simbólico) y representación para el caso específico de clasificación binaria y espacios de características de 2 o 3 dimensiones:**

- **Función** `funcion_calcula_d1_d2_d12_LDA_2Clases_2_3_Dim`:

Es una función específica para conjuntos de datos X-Y compuestos por 2 o 3 variables de entrada y 2 clases. La función calcula las funciones de decisión `d1`, `d2`, y la función discriminante, `d12`, para una instancia X definida de forma simbólica. Al igual que en el caso del clasificador QDA, la frontera de separación del clasificador LDA también se puede representar mediante las funciones `fimplicit` o `fimplicit3`, según se trabaje en dos o tres dimensiones, de Matlab a partir de la variable simbólica `d12`.

```
[d1, d2, d12] = funcion_calcula_d1_d2_d12_LDA_2Clases_2_3_Dim (X, Y)
```

CLASIFICADOR k-NN

Entrada: conjunto de instancias prototipo de las que ya se conoce su clase (conjunto de entrenamiento X_{Train} - Y_{Train}), instancia cuya clase se pretende predecir (X_{Test}) y parámetro k (ver explicación a continuación).

Salida: codificación de la clase predicha para la instancia de test.

Principio de funcionamiento: el clasificador calcula las k instancias del conjunto de entrenamiento (X_{Train}) que presentan más similitud con cada instancia de test (X_{Test}). Se predice que la clase de cada instancia de X_{Test} (variable de salida Y_{Test}) es la clase más numerosa de estas “ k ” instancias más parecidas. El concepto de similitud puede implementarse por medio de cualquier función distancia. En nuestro caso, emplearemos distancia Euclídea.

```
YTest = funcion_knn (XTest, XTrain, YTrain, k)
```

GUÍA DE PROGRAMACIÓN:

```
% CLASIFICADOR K-NEAREST-NEIGHBOURS: básicamente consiste en medir la
% distancia entre la muestra desconocida y cada una de las
% muestras de entrenamiento disponibles. Nos quedamos con las k distancias
% menores y asociamos a la muestra desconocida aquella clase que más se
% repita.
```

% ENTRADAS-SALIDAS DE LA FUNCIÓN

```
% XTest: matriz numMuestrasTest x numDescriptores

% XTrain: matriz numMuestrasTrain x numDescriptores
% YTrain: matriz numMuestrasTrain x 1 (codificación de las clases
% para cada instancia de train)

% k: número de "vecinos" más cercanos considerado

% YTest: matriz numMuestrasTest x 1 (codificación de las clases predichas
% para cada instancia de test).
```

% FUNDAMENTOS DE PROGRAMACIÓN:

% POR CADA MUESTRA DE TEST:

% 1.- CALCULO DEL VECTOR DISTANCIAS ENTRE LA INSTANCIA DE TEST Y TODAS LAS
% INSTANCIAS DE TRAIN

% Como la distancia que empleamos es la Euclídea, emplearemos notación
% matricial para el cálculo de distancia entre muestras manejando vectores
% columna. Para ello, se utilizará repmat para más eficiencia y programarlo
% en una/dos líneas de código, sin necesidad de bucle

% 2.- LOCALIZAR LAS k INSTANCIAS DE XTrain MAS CERCANAS A LA INSTANCIA DE
% TEST BAJO CONSIDERACIÓN

% 3.- CREAR UN VECTOR CON LAS CODIFICACIONES DE LAS CLASES DE ESAS
% k-INSTANCIAS MÁS CERCANAS

% 4.- ANALIZAR ESE VECTOR PARA CONTAR EL NÚMERO DE VECES QUE APARECE
% CADA CODIFICACIÓN PRESENTE EN EL VECTOR (unique del vector)

% 5.- EL VALOR DE YTEST EN LA POSICIÓN CORRESPONDIENTE A LA INSTANCIA DE
% XTEST QUE SE ESTÁ ANALIZANDO ES LA CODIFICACIÓN DE LA CLASE MÁS NUMEROSA.

% - SI HAY MÁS DE UNA CLASE CON EL NÚMERO MÁXIMO DE VOTOS, DEVOLVER LA
% CLASE DE LA INSTANCIA MÁS CERCANA A LA DE TEST (ENTRE ESAS INSTANCIAS
% DE LAS CLASES MÁS NUMEROSAS)